

# Laboratorio 8 - Sistema de Recompensas (Arquitectura Hexagonal + EDA)

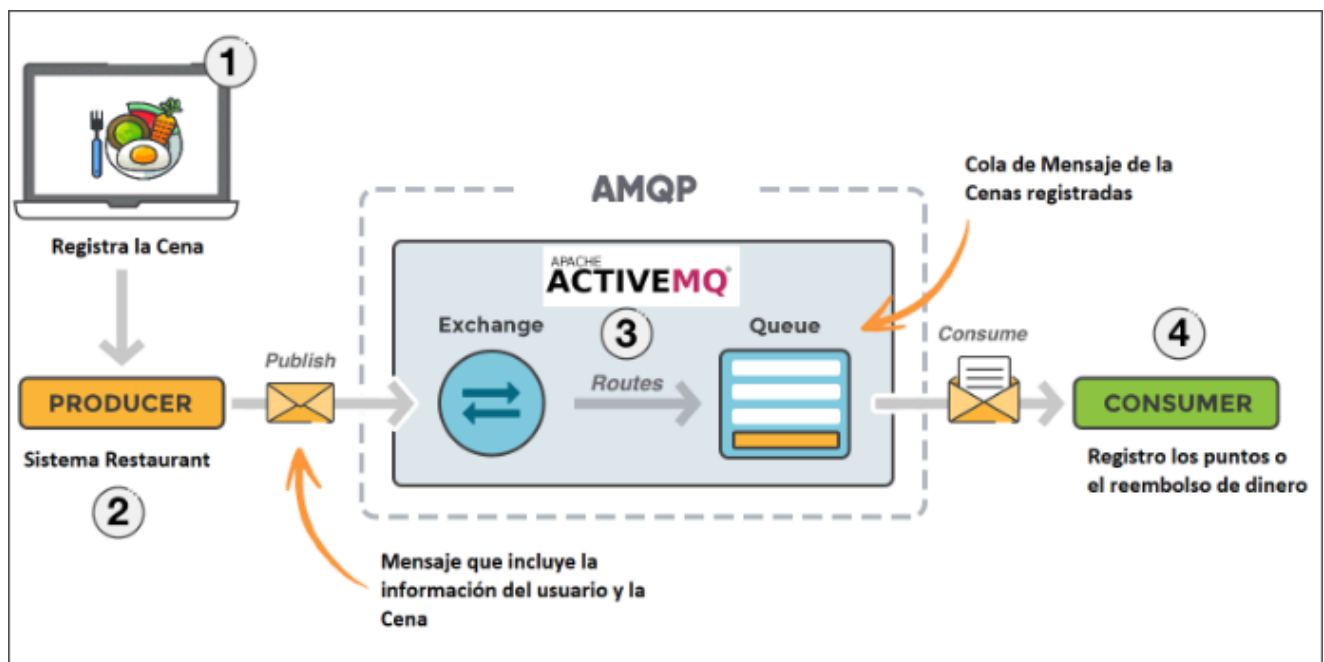
Sistema de recompensas para restaurantes implementado con **Arquitectura Hexagonal (Ports & Adapters)** y **Event-Driven Architecture** sobre **RabbitMQ**.

- **Repositorio:** [https://github.com/EV081/Laboratorio\\_8\\_software](https://github.com/EV081/Laboratorio_8_software)
- **Análisis SonarQube:** [https://sonarqube.ingsoftware.lat/dashboard?id=Elmer\\_Villegas\\_t1&codeScope=overall](https://sonarqube.ingsoftware.lat/dashboard?id=Elmer_Villegas_t1&codeScope=overall)

## 1. Problema

Cada vez que un cliente cena en un restaurante afiliado, una parte de su consumo se convierte en puntos que se acreditan a su cuenta. El sistema debe procesar grandes volúmenes de transacciones en tiempo real, por lo que se desacopla mediante mensajería asíncrona.

Flujo (Figura 1 del enunciado):



## 2. Arquitectura

El problema combina dos exigencias: el negocio necesita procesar muchas cenas en tiempo real sin que un servicio lento (el envío de correo) frene al resto, y al mismo tiempo la lógica de puntos debe poder evolucionar sin reescribir la infraestructura. **EDA con RabbitMQ** resuelve lo primero porque desacopla los servicios en el tiempo: quien registra la cena no espera a que se calculen puntos ni a que salga el email, solo publica un evento y sigue. **La Arquitectura Hexagonal** resuelve lo segundo

porque aísla el dominio (reglas) de los detalles técnicos (RabbitMQ, FastAPI, persistencia), de modo que cambiar de broker, de base de datos o de proveedor de email no toca la lógica de negocio, solo el adaptador correspondiente.

## Implementación

La solución que se implementó fue con tres microservicios que se comunican de forma asíncrona a través de RabbitMQ, siguiendo un estilo **Event-Driven**: el módulo de Restaurantes recibe las cenas por una API REST y publica el evento `dinner.registered`; el módulo de Reward lo consume para calcular los puntos y a su vez emite `reward.processed`; y el módulo de Notificaciones reacciona a ese último para avisarle al cliente. De esta forma cada servicio se despliega y escala por separado, y ninguno depende directamente de los otros, solo del broker.

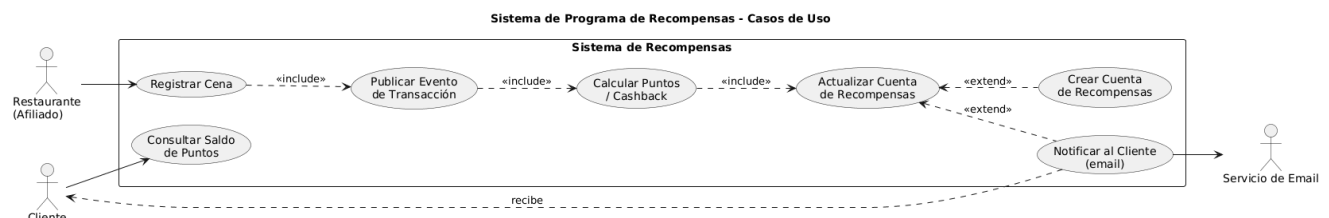
Dentro de cada microservicio aplicamos el patrón **Hexagonal (Ports & Adapters)**, que organiza el código en tres capas con dependencias hacia adentro:

- **Domain**: Entidades, *value objects*, políticas y **puertos** (interfaces abstractas). No depende de nada externo.
- **Application**: Casos de uso que orquestan el dominio. Depende solo de puertos.
- **Infrastructure**: Adaptadores concretos (RabbitMQ, REST con FastAPI, persistencia en memoria). Implementan los puertos del dominio.

Esto garantiza:

| Atributo          | Cómo se cumple                                                |
|-------------------|---------------------------------------------------------------|
| Alta cohesión     | Cada módulo tiene una sola responsabilidad                    |
| Bajo acoplamiento | El dominio no conoce a RabbitMQ; los adaptadores se inyectan  |
| Modularidad       | 3 servicios independientes, cada uno desplegable por separado |
| Escalabilidad     | Cualquier servicio se puede replicar horizontalmente          |
| Event-Driven      | Servicios se comunican solo vía eventos por el broker         |

## Diagrama de Casos de Uso



## Explicación del diagrama

Identificamos tres actores. El Restaurante es el actor primario que dispara todo el flujo cuando registra una cena; el Cliente es primario cuando consulta su saldo y aparece como receptor cuando

le llega una notificación; y el Servicio de Email es un actor secundario, un proveedor externo al que le delegamos el envío real del correo.

El flujo principal arranca con Registrar Cena (UC1), que el restaurante invoca a través de la API REST. Este caso siempre incluye Publicar Evento de Transacción(UC2), porque el registro no termina hasta que el evento se publica en RabbitMQ. UC2 a su vez incluye Calcular Puntos / Cashback (UC3), que aplica la política configurada (por defecto, 10 % del consumo), y este se incluye dentro de Actualizar Cuenta de Recompensas (UC4), que persiste los puntos en la cuenta del cliente.

Dos casos extienden UC4 porque solo se ejecutan bajo cierta condición. Crear Cuenta de Recompensas (UC7) se dispara únicamente cuando la tarjeta aparece por primera vez en el sistema, es decir, cuando no existe una cuenta previa asociada a ese card\_number. Notificar al Cliente por email (UC5) se modela como extensión porque, además de depender de la actualización exitosa de la cuenta, requiere salir del sistema para llegar al proveedor externo de correo.

Por último, Consultar Saldo de Puntos (UC6) es un flujo independiente que el cliente ejerce por su cuenta, sin estar encadenado al registro de cenas.

El actor secundario **"Servicio de Email"** representa un proveedor externo (ej. SendGrid, AWS SES) al que el sistema delega el envío real del correo. La implementación actual simula este envío, pero el puerto `NotificationSender` permite sustituir el adaptador sin modificar el dominio ni la capa de aplicación, manteniendo la arquitectura hexagonal.

### 3. Mensajería

| Cola RabbitMQ                  | Productor                       | Consumidor                         | Payload JSON                                                     |
|--------------------------------|---------------------------------|------------------------------------|------------------------------------------------------------------|
| <code>dinner.registered</code> | <code>restaurant_service</code> | <code>rewards_service</code>       | <code>{amount, card_number, restaurant_code, occurred_at}</code> |
| <code>reward.processed</code>  | <code>rewards_service</code>    | <code>notifications_service</code> | <code>{card_number, points_added, total_points}</code>           |

**Política de puntos** ( `PercentagePointsPolicy` ): 10 % del monto consumido por defecto.

### 4. Cómo correr

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

### Tests

```
pytest
```

## Levantar los servicios (3 terminales)

```
# Terminal 1 – Restaurante (API REST)
python -m restaurant_service.main

# Terminal 2 – Recompensas
python -m rewards_service.main

# Terminal 3 – Notificaciones
python -m notifications_service.main
```

## Enviar una cena de prueba

```
curl -X POST http://localhost:8000/dinners \
-H 'Content-Type: application/json' \
-d '{
  "amount": "150.00",
  "card_number": "411111111111234",
  "restaurant_code": "REST-001",
  "occurred_at": "2026-05-24T19:30:00"
}'
```

## Análisis SonarQube

Si tienes `sonar-scanner` instalado en tu sistema:

```
sonar-scanner
```

## 5. Justificación del broker

---

Se eligió **RabbitMQ** sobre Kafka/ActiveMQ porque:

1. La Figura 1 se muestra ActiveMQ (modelo AMQP); RabbitMQ usa el mismo modelo `Producer -> Exchange -> Queue -> Consumer`.
2. El caso de uso es *work queue* transaccional, no streaming masivo (Kafka sería sobredimensionar).
3. Gracias a hexagonal, migrar a Kafka requiere reemplazar `rabbit_consumer.py` / `rabbit_publisher.py` — el resto del código no cambia.

## 6. Evidencia de ejecución de pruebas y Calidad

---

Ejecución de las pruebas automatizadas con `pytest` :

```
pytest
===== test session starts =====
platform linux -- Python 3.13.12, pytest-9.0.3, pluggy-1.6.0 -- /home/vssz/UTEC/5to/software/Laboratorio_8_software/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/vssz/UTEC/5to/software/Laboratorio_8_software
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0, anyio-4.13.0
collected 55 items

tests/notifications_service/test_console_sender.py::test_console_sender_writes_message PASSED [ 1%]
tests/notifications_service/test_console_sender.py::test_console_sender_uses_print_by_default PASSED [ 3%]
tests/notifications_service/test_notification.py::test_notification_message_includes_points PASSED [ 5%]
tests/notifications_service/test_notification.py::test_notification_masks_card_number PASSED [ 7%]
tests/notifications_service/test_notification.py::test_short_card_is_not_masked PASSED [ 9%]
tests/notifications_service/test_notification.py::test_rejects_empty_card PASSED [ 10%]
tests/notifications_service/test_notification.py::test_rejects_negative_points_added PASSED [ 12%]
tests/notifications_service/test_notification.py::test_rejects_negative_total_points PASSED [ 14%]
tests/notifications_service/test_rabbit_consumer.py::test_on_message_deserializes_and_sends PASSED [ 16%]
tests/notifications_service/test_rabbit_consumer.py::test_start_wires_consumption PASSED [ 18%]
tests/notifications_service/test_send_notification.py::test_use_case_delegates_to_sender PASSED [ 20%]
tests/restaurant_service/test_api.py::test_post_dinners_returns_202_and_publishes PASSED [ 21%]
tests/restaurant_service/test_api.py::test_post_dinners_rejects_negative_amount PASSED [ 23%]
tests/restaurant_service/test_api.py::test_post_dinners_rejects_missing_fields PASSED [ 25%]
tests/restaurant_service/test_api.py::test_health_endpoint PASSED [ 27%]
tests/restaurant_service/test_api.py::test_post_dinners_returns_400_when_domain_rejects_whitespace_card PASSED [ 29%]
tests/restaurant_service/test_dinner.py::test_dinner_is_created_with_valid_fields PASSED [ 30%]
tests/restaurant_service/test_dinner.py::test_dinner_is_immutable PASSED [ 32%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_non_positive_amount[amount0] PASSED [ 34%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_non_positive_amount[amount1] PASSED [ 36%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_empty_card_number[] PASSED [ 38%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_empty_card_number[ ] PASSED [ 40%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_empty_restaurant_code[] PASSED [ 41%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_empty_restaurant_code[ ] PASSED [ 43%]
tests/restaurant_service/test_dinner.py::test_dinner_rejects_invalid_occurred_at PASSED [ 45%]
tests/restaurant_service/test_rabbit_publisher.py::test_publisher_serializes_dinner_and_calls_basic_publish PASSED [ 47%]
tests/restaurant_service/test_rabbit_publisher.py::test_publisher_closes_connection_even_on_failure PASSED [ 49%]
tests/restaurant_service/test_register_dinner.py::test_use_case_publishes_dinner_through_port PASSED [ 50%]
tests/restaurant_service/test_register_dinner.py::test_use_case_handles_multiple_dinners PASSED [ 52%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_is_created_with_valid_fields PASSED [ 54%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_rejects_non_positive_amount[amount0] PASSED [ 56%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_rejects_non_positive_amount[amount1] PASSED [ 58%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_rejects_empty_card PASSED [ 60%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_rejects_empty_restaurant PASSED [ 61%]
tests/rewards_service/test_dinner_event.py::test_dinner_event_rejects_invalid_datetime PASSED [ 63%]
tests/rewards_service/test_in_memory_repo.py::test_find_returns_none_for_unknown_card PASSED [ 65%]
tests/rewards_service/test_in_memory_repo.py::test_save_then_find_returns_same_account PASSED [ 67%]
tests/rewards_service/test_in_memory_repo.py::test_save_overwrites_previous_account PASSED [ 69%]
tests/rewards_service/test_in_memory_repo.py::test_save_overwrites_previous_account PASSED [ 69%]
tests/rewards_service/test_points_policy.py::test_default_policy_gives_10_percent PASSED [ 70%]
tests/rewards_service/test_points_policy.py::test_custom_percentage_is_applied PASSED [ 72%]
tests/rewards_service/test_points_policy.py::test_policy_rounds_to_two_decimals PASSED [ 74%]
tests/rewards_service/test_points_policy.py::test_policy_rejects_negative_amount PASSED [ 76%]
tests/rewards_service/test_points_policy.py::test_policy_rejects_non_positive_percentage PASSED [ 78%]
tests/rewards_service/test_process_dinner.py::test_process_dinner_creates_account_when_missing PASSED [ 80%]
tests/rewards_service/test_process_dinner.py::test_process_dinner_accumulates_on_existing_account PASSED [ 81%]
tests/rewards_service/test_process_dinner.py::test_process_dinner_publishes_notification PASSED [ 83%]
tests/rewards_service/test_rabbit_consumer.py::test_on_message_processes_valid_event PASSED [ 85%]
tests/rewards_service/test_rabbit_consumer.py::test_start_registers_consumer_and_starts_consuming PASSED [ 87%]
tests/rewards_service/test_rabbit_notification_publisher.py::test_publishes_payload_to_queue PASSED [ 89%]
tests/rewards_service/test_rabbit_notification_publisher.py::test_connection_is_closed_on_error PASSED [ 90%]
tests/rewards_service/test_reward_account.py::test_account_starts_with_zero_points PASSED [ 92%]
tests/rewards_service/test_reward_account.py::test_account_accumulates_points PASSED [ 94%]
tests/rewards_service/test_reward_account.py::test_account_rejects_empty_card_number PASSED [ 96%]
tests/rewards_service/test_reward_account.py::test_account_rejects_negative_initial_points PASSED [ 98%]
tests/rewards_service/test_reward_account.py::test_add_points_rejects_negative_amount PASSED [ 100%]

===== tests coverage =====
coverage: platform linux, python 3.13.12-final-0

Name Stmts Miss Cover Missing
-----
notifications_service/application/send_notification.py 7 0 100%
notifications_service/domain/model/notification.py 20 0 100%
notifications_service/domain/ports/notification_sender.py 5 0 100%
notifications_service/infrastructure/messaging/rabbit_consumer.py 16 0 100%
notifications_service/infrastructure/sender/console_sender.py 8 0 100%
restaurant_service/application/register_dinner.py 7 0 100%
restaurant_service/domain/model/dinner.py 18 0 100%
restaurant_service/domain/ports/dinner_publisher.py 5 0 100%
restaurant_service/infrastructure/messaging/rabbit_publisher.py 7 0 100%
restaurant_service/infrastructure/rest/api.py 25 0 100%
rewards_service/application/process_dinner.py 19 0 100%
rewards_service/domain/model/dinner_event.py 18 0 100%
rewards_service/domain/model/reward_account.py 15 0 100%
rewards_service/domain/policy/points_policy.py 14 0 100%
rewards_service/domain/ports/notification_publisher.py 5 0 100%
rewards_service/domain/ports/reward_repository.py 8 0 100%
rewards_service/infrastructure/messaging/rabbit_consumer.py 17 0 100%
rewards_service/infrastructure/messaging/rabbit_notification_publisher.py 18 0 100%
rewards_service/infrastructure/persistence/in_memory_repo.py 10 0 100%
shared/messaging/rabbit_base.py 35 0 100%
TOTAL 277 0 100%
Coverage XML written to file coverage.xml

===== 55 passed in 0.56s =====
```

Cobertura y métricas de calidad reportadas por SonarQube:



Quality Gate ⓘ

Passed

⚠ The last analysis has warnings. [See details](#)

New Code

Overall Code

Security

0 Open issues

A

Reliability

0 Open issues

A

Maintainability

0 Open issues

A

Accepted issues

0

Valid issues that were not fixed

⌚

Coverage

100%

Required  $\geq$  80.0%  
On 277 lines to cover.



Duplications

0.0%

Required  $\leq$  2.0%  
On 726 lines.



Security Hotspots

0

A